



EC-200 Data Structures

LAB MANUAL # 14

Course Instructor: Dr. Anum Abdul Salam

Lab Engineer: Engineer Hira Irshad

Student Name: _____

Degree/ Syndicate: _____

	Trait	Obtained Marks	Maximum Marks
R1	Application Functionality 20%		20
R2	Specification & Data structure implementation 30%		30
R3	Reusability 10%		10
R4	Input Validation 10%		10
R5	Efficiency 20%		20
R6	Delivery 10%		10
R7	Plagiarism above 60%		0
Total			100

Total Marks = Obtained Marks ($\sum_{i=1}^6 R_i * R_7$)

Lab # 14: Implementation of B-Trees and AVL Trees in C++

Lab Objective:

To implement B-Trees and AVL Trees in C++

Lab Description:

Balanced trees are a type of binary or multi-way search tree that maintain their height to ensure operations like search, insert, and delete run in **$O(\log n)$** time.

- **AVL Tree:**

A self-balancing binary search tree where the difference between heights of left and right subtrees (balance factor) is at most 1. Rotations (LL, RR, LR, RL) are used to maintain balance.

- **B-Tree:**

A generalized multi-way search tree optimized for systems that read and write large blocks of data. Each node can have multiple keys and children, making it suitable for databases and file systems.

AVL Trees:

In AVL trees, the difference between the heights of left and right subtrees, known as the **Balance Factor**, must be at most one. Once the difference exceeds one, the tree automatically executes the balancing algorithm until the difference becomes one again.

$$\text{BALANCE FACTOR} = \text{HEIGHT}(\text{LEFT SUBTREE}) - \text{HEIGHT}(\text{RIGHT SUBTREE})$$

There are usually four cases of rotation in the balancing algorithm of AVL trees: LL, RR, LR, RL.

Basic Operations of AVL Trees

The basic operations performed on the AVL Tree structures include all the operations performed on a binary search tree, since the AVL Tree at its core is actually just a binary search tree holding all its properties. Therefore, basic operations performed on an AVL Tree are **Insertion** and **Deletion**.

Insertion operation

The data is inserted into the AVL Tree by following the Binary Search Tree property of insertion, i.e. the left subtree must contain elements less than the root value and right subtree must contain all the greater elements.

However, in AVL Trees, after the insertion of each element, the balance factor of the tree is checked; if it does not exceed 1, the tree is left as it is. But if the balance factor exceeds 1, a

balancing algorithm is applied to readjust the tree such that balance factor becomes less than or equal to 1 again.

Deletion operation

Deletion in the AVL Trees take place in three different scenarios:

- **Scenario 1 (Deletion of a leaf node)** – If the node to be deleted is a leaf node, then it is deleted without any replacement as it does not disturb the binary search tree property. However, the balance factor may get disturbed, so rotations are applied to restore it.
- **Scenario 2 (Deletion of a node with one child)** – If the node to be deleted has one child, replace the value in that node with the value in its child node. Then delete the child node. If the balance factor is disturbed, rotations are applied.
- **Scenario 3 (Deletion of a node with two child nodes)** – If the node to be deleted has two child nodes, find the inorder successor of that node and replace its value with the inorder successor value. Then try to delete the inorder successor node. If the balance factor exceeds 1 after deletion, apply balance algorithms.

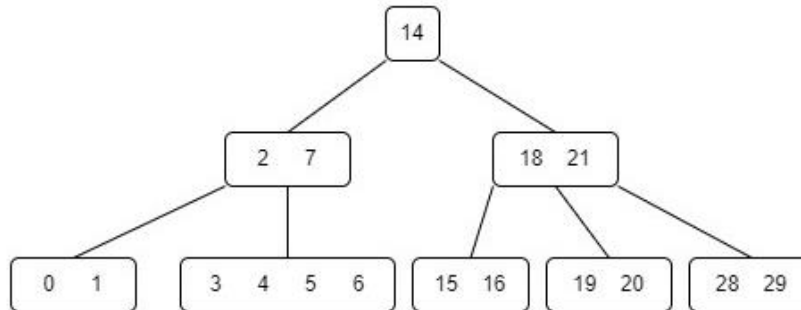
B-Trees:

B trees are extended binary search trees that are specialized in m-way searching, since the order of B trees is 'm'. Order of a tree is defined as the maximum number of children a node can accommodate. Therefore, the height of a b tree is relatively smaller than the height of AVL tree and RB tree.

They are general form of a Binary Search Tree as it holds more than one key and two children.

The various properties of B trees include:

- Every node in a B Tree will hold a maximum of m children and (m-1) keys, since the order of the tree is m.
- Every node in a B tree, except root and leaf, can hold at least $m/2$ children
- The root node must have no less than two children.
- All the paths in a B tree must end at the same level, i.e. the leaf nodes must be at the same level.
- A B tree always maintains sorted data.



B trees are also widely used in disk access, minimizing the disk access time since the height of a b tree is low.

Basic Operations of B Trees

The operations supported in B trees are Insertion, deletion and searching with the time complexity of $O(\log n)$ for every operation.

Insertion operation

The insertion operation for a B Tree is done similar to the Binary Search Tree but the elements are inserted into the same node until the maximum keys are reached. The insertion is done using the following procedure –

Step 1 – Calculate the maximum and, minimum number of keys a node can hold, where m is denoted by the order of the B Tree.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree

- Order (m) = 4
- Maximum Keys ($m - 1$) = 3
- Minimum Keys $\left(\left\lceil \frac{m}{2} \right\rceil\right) - 1 = 1$
- Maximum Children = 4
- Minimum Children $\left(\left\lceil \frac{m}{2} \right\rceil\right) = 2$

Step 2 – The data is inserted into the tree using the binary search insertion and once the keys reach the maximum number, the node is split into half and the median key becomes the internal node while the left and right keys become its children.

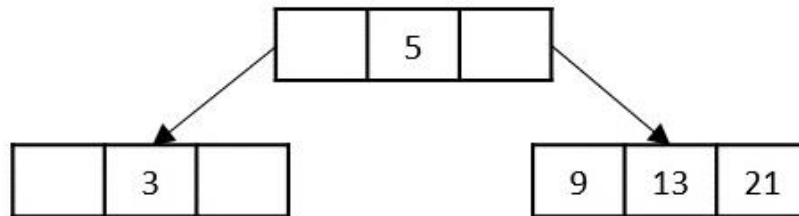
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 9 will cause overflow in the node; hence it must be split.

Step 3 – All the leaf nodes must be on the same level.

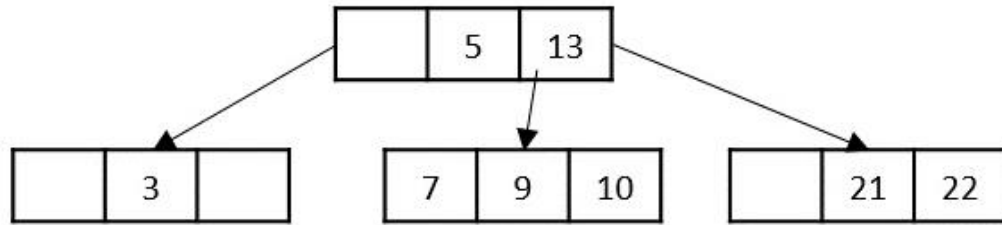
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 22 will cause overflow in the node; hence it must be split.

The keys, 5, 3, 21, 9, 13 are all added into the node according to the binary search property but if we add the key 22, it will violate the maximum key property. Hence, the node is split in half, the median key is shifted to the parent node and the insertion is then continued.

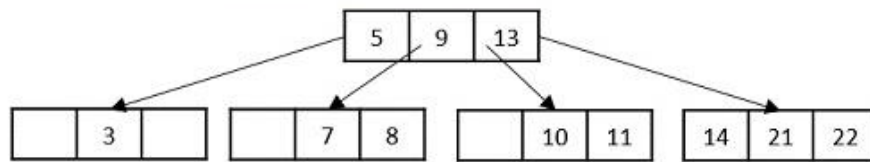
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 11 will cause overflow in the node; hence it must be split.

Another hiccup occurs during the insertion of 11, so the node is split and median is shifted to the parent.

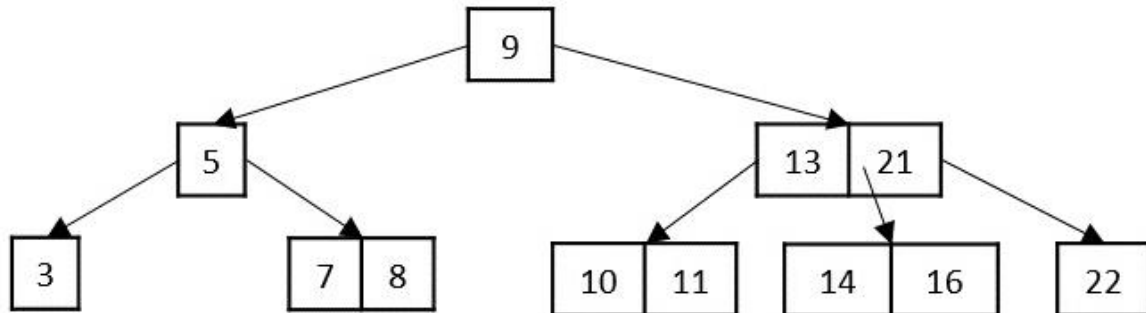
Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



Adding 16 will cause overflow in the node; hence it must be split.

While inserting 16, even if the node is split in two parts, the parent node also overflows as it reached the maximum keys. Hence, the parent node is split first and the median key becomes the root. Then, the leaf node is split in half the median of leaf node is shifted to its parent.

Insert 5, 3, 21, 9, 13, 22, 7, 10, 11, 14, 8, 16 into a B tree



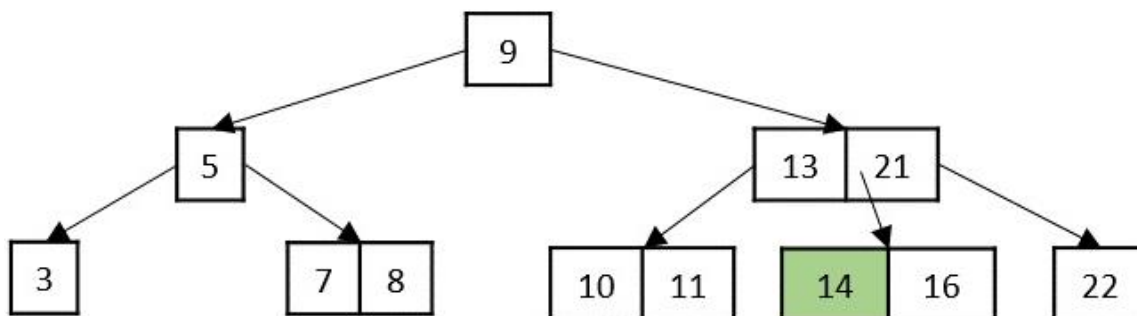
The final B tree after inserting all the elements is achieved.

Deletion operation

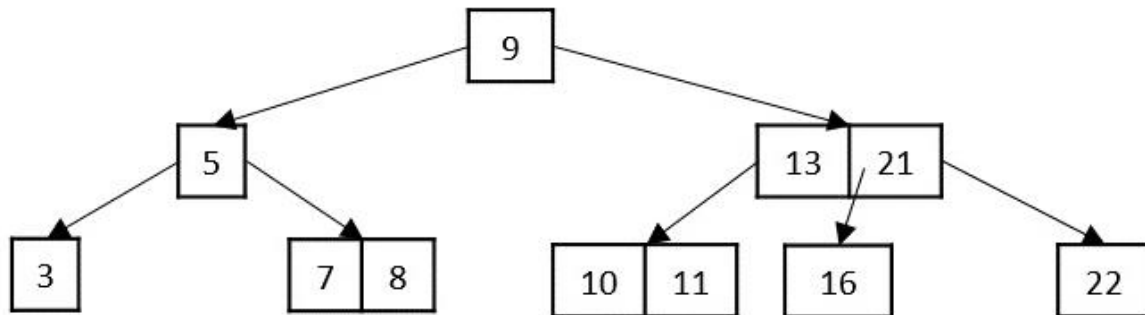
The deletion operation in a B tree is slightly different from the deletion operation of a Binary Search Tree. The procedure to delete a node from a B tree is as follows –

Case 1 – If the key to be deleted is in a leaf node and the deletion does not violate the minimum key property, just delete the node.

Delete key 14

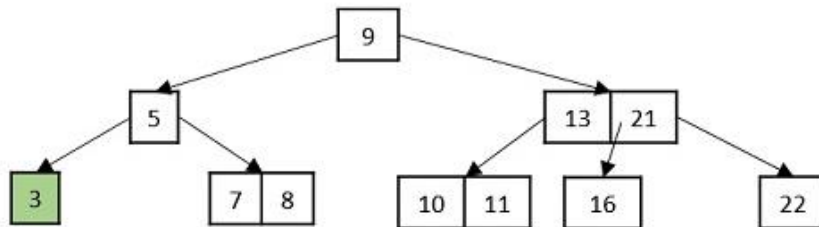


Delete key 14

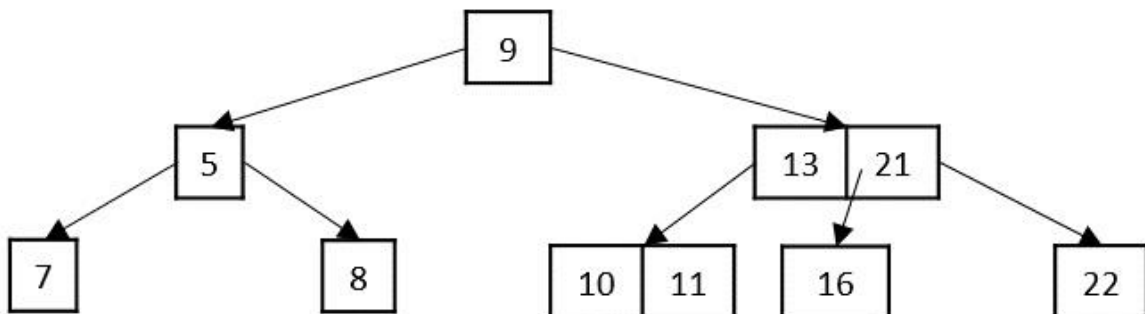


Case 2 – If the key to be deleted is in a leaf node but the deletion violates the minimum key property, borrow a key from either its left sibling or right sibling. In case if both siblings have exact minimum number of keys, merge the node in either of them.

Delete key 3

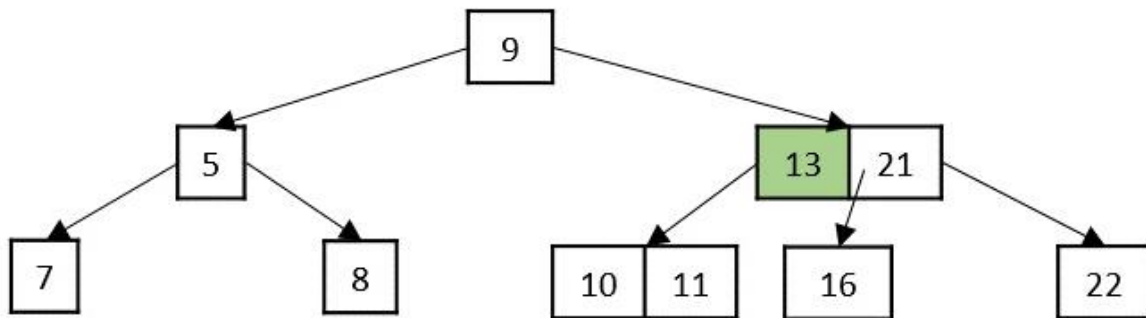


Delete key 3

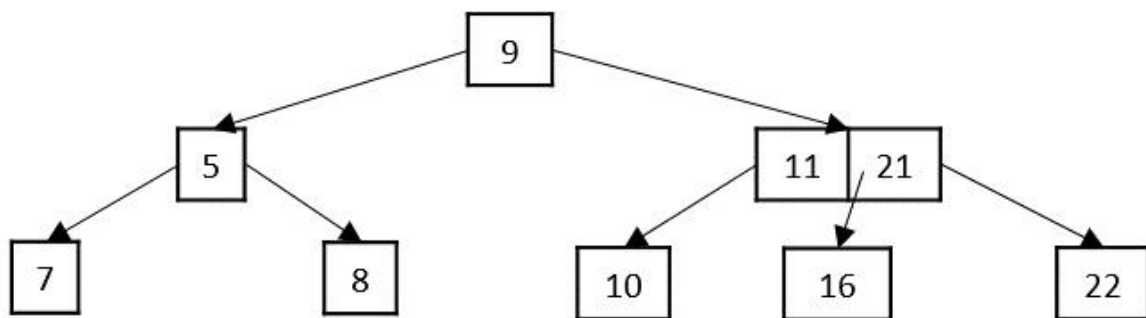


Case 3 – If the key to be deleted is in an internal node, it is replaced by a key in either left child or right child based on which child has more keys. But if both child nodes have minimum number of keys, they're merged together.

Delete key 13

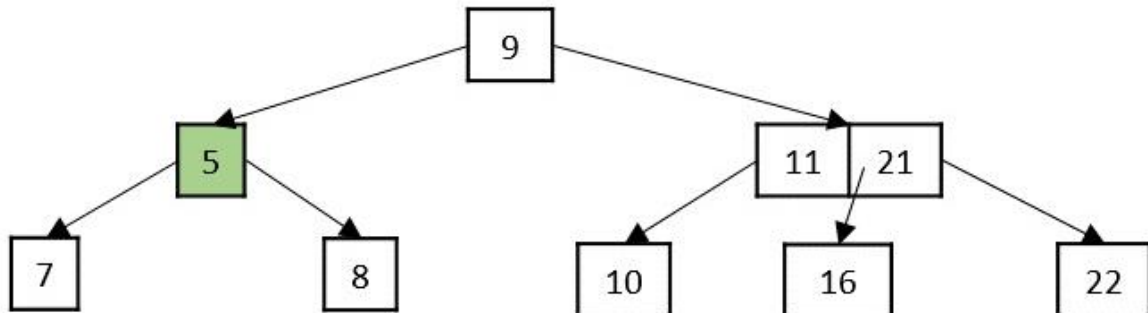


Delete key 13

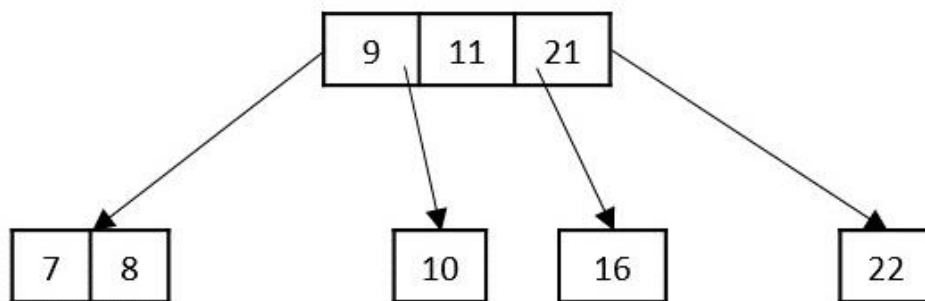


Case 4 – If the key to be deleted is in an internal node violating the minimum keys property, and both its children and sibling have minimum number of keys, merge the children. Then merge its sibling with its parent.

Delete key 5



Delete key 5



Lab Tasks:

1. Implement an AVL Tree ADT in C++ that supports insertion, deletion, and search operations on a AVL Tree. Ensure the tree remains balanced after each update by applying appropriate rotations (LL, RR, LR, RL). Provide traversal functions (inorder, preorder, or postorder) to display the tree contents.

Test your AVL Tree ADT by executing the provided test cases in main function.

Test Plan:

Test Case #	Operation	Input Data	Expected Output / Behavior
1	Insert	10, 20, 30	
2	Insert	30, 20, 10	
3	Insert	30, 10, 20	
4	Insert	10, 30, 20	

5	Search	20	
6	Search	100	
7	Delete	20	
8	Traversal	10, 20, 30, 40, 50	
9	Delete Root	Root node	

2. Implement a B-Tree ADT in C++ with a specified minimum degree t . The tree should support insertion, deletion, and search operations, including node splitting and merging where necessary. Provide a traversal function to display the keys in sorted order.

Test your B-Tree ADT by executing the provided test cases in main function.

Test Plan:

Test Case #	Operation	Input Data	Expected Output / Behavior
1	Insert	Keys in ascending order	
2	Insert	Keys in random order	
3	Search	Existing key	
4	Search	Non-existing key	
5	Insert	Large number of keys	
6	Delete	Leaf node key	
7	Delete	Internal node key	
8	Delete	Causing underflow	
9	Traversal	Multiple inserted keys	